

Astroinformatics 1: Graded Practice 2

Juan David Galan Vargas*
Universidad de Antofagasta, Chile.
 (Dated: June 9, 2026)

I. TASK 1

On this first task we had to work with the CSV files generated on the last graded practice. Lets remember that there were 20 files in total,all of which came originally from FITS files. The objective was to write a Shell Script that would perform the following modification to the CSV files:

1. Change their delimiter from "," to " "
2. Change their extension from ".csv" to ".lc"
3. Remove all columns which were not part of the light curve. Therefore, it was decided to keep the following columns as to keep the most amount of related data:
 - TIME
 - TIMECORR
 - SAP FLUX
 - SAP FLUX ERR
 - PDCSAP FLUX
 - PDCSAP FLUX ERR

The shell script used was as follows:

```
SOURCE_DIR="[Insert_here_the_path]"
for file in "$SOURCE_DIR"/*.csv; do
    [ -f "$file" ] ||
continue
```

```
#Change of extension
base=$(basename "$file" .csv)
output="./${base}.lc"
```

```
#Change of delimiter
awk -F',', ' '
```

```
#Filtering Columns
NR==1
{for (i=1;i<=NF;i++)
    gsub(/^[ \t "]+|[ \t "]+$/, "", $i)
#Go_line_by_line_in_the_CSV_File
```

```

_____if_($i=="TIME")_____c1=i
_____if_($i=="TIMECORR")_____c2=i
_____if_($i=="SAP_FLUX")_____c3=i
_____if_($i=="SAP_FLUX_ERR")_____c4=i
_____if_($i=="PDCSAP_FLUX")_____c5=i
_____if_($i=="PDCSAP_FLUX_ERR")_c6=i
_____}
_____print_"TIME_TIMECORR
_____SAP_FLUX_SAP_FLUX_ERR
_____PDCSAP_FLUX_PDCSAP_FLUX_ERR"
_____next
_____}

{print_$c1,_$c2,_$c3,_$c4,_$c5,_$c6
#Line_by_line, _printing_the_selected_columns
}
' "$file" > "$output"
echo "Created_$output"
done

```

As we can see on the previous code, the first section looks for all CSV files on the specified directory. Then, it takes the file and creates a copy on which the extension is changed, it also uses the AWK command to detect the delimiters in the file and replace them. The last part focuses on going line by line and checking if some data is part of our columns of interest saving it into a variable if so. Following this, it prints those variables in the new file (with the changed extension). Finally, it also prints on the terminal the name of all the new files created. We can see the Shell Script in action on the following screen-shots:

```

Created /test202334/045131-0073-0000000000001750268-0268-5-1.c-1c
Created /test202334/045131-0073-000000000000175406-0268-5-1.c-1c
Created /test202334/045131-0073-000000000000187774-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000001820628-0268-5-1.c-1c
Created /test202334/045131-0073-000000000000184666-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000001924216-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000001912909-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000001947463-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000001959736-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000001958983-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000002000694-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000002008765-0268-5-1.c-1c
Created /test202334/045131-0073-000000000000201491-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000002103239-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000002104696-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000002105589-0268-5-1.c-1c
Created /test202334/045131-0073-000000000000214357-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000002149979-0268-5-1.c-1c
Created /test202334/045131-0073-000000000000215241-0268-5-1.c-1c
Created /test202334/045131-0073-0000000000002234692-0268-5-1.c-1c

```

Figure 1. Output of the Shell Script in the terminal after running it.

* Institutional e-mail: JUAN.GALAN.VARGAS@ua.cl

```

jdg@jdgpc:~/Documents/Tareas_ID/Astrofagasta/Magister/Materias/1er Semestre/Astroinformatica I/Practice_1$ python3 task1.py
print("Your star may
be a Wolf-Rayet Star
(TOO HOT! )")

```

Figure 2. Working directory after running the Shell Script with the new files.

```

tess2023341045131-s0073-0000000002234692-0268-s_lc.lc
TIME TIMECORR SAP_FLUX SAP_FLUX_ERR PDCSAP_FLUX PDCSAP_FLUX_ERR
3285.798991170703 0.005529689
3285.800380063614 0.005529694
3285.80176895699 0.005529699
3285.8031578503674 0.005529704
3285.8045467437446 0.005529709 1558.0974 6.226424 1215.4843 8.989949
3285.805935637121 0.0055297143 1560.7036 6.231727 1216.2783 8.997606
3285.8073245300325 0.005529719 1550.258 6.22222 1204.2166 8.98388
3285.8087134234097 0.005529724 1556.43 6.2244453 1210.0746 8.987093
3285.810102316786 0.005529729 1557.2596 6.2280173 1221.4556 8.9922495
3285.8114912101632 0.0055297343 1545.991 6.222984 1200.1544 8.984982

```

Figure 3. New ".lc" file content with the expected changes.

II. TASK 2

The next task focused on coding a star spectral type classifier based only on their temperature (in Kelvin) as input, specifically, this program had to take the input as a command line argument when invoked in the terminal. The following Python code was used for this task:

```

import sys
#Temperature in Kelvin
t_k = float(sys.argv[1])

if t_k > 30000 and t_k <=60000:
    print("The_Spectral_Type
of_your_Star_is:_O")
elif t_k > 10000 and t_k<=30000:
    print("The_Spectral_Type
of_your_Star_is:_B")
elif t_k > 7500 and t_k<=10000:
    print("The_Spectral_Type
of_your_Star_is:_A")
elif t_k > 6000 and t_k<=7500:
    print("The_Spectral_Type
of_your_Star_is:_F")
elif t_k > 5000 and t_k<=6000:
    print("The_Spectral_Type
of_your_Star_is:_G")
elif t_k > 3500 and t_k<=5000:
    print("The_Spectral_Type
of_your_Star_is:_K")
elif t_k >= 2000 and t_k<=3500:
    print("The_Spectral_Type
of_your_Star_is:_M")
elif t_k < 2000:
    print("Your_Star_is
a_Brown_Dwarf")
else:

```

```

print("Your_star_may
be_a_Wolf-Rayet_Star
(TOO_HOT! )")

```

Notice that for the lower end of the temperature, the class "Brown Dwarf" was assigned, as those objects are too small to sustain the nuclear fusion of regular hydrogen, but dense and hot enough to briefly fuse deuterium. On the other hand, for the upper end of temperature the class "Wolf-Rayet Star" was assigned, which are rare kind of stars that can have higher temperature than most O-type stars. On the following screen-shoot we can see the program in action:

```

jdg@jdgpc:~/Documents/Tareas_ID/Astrofagasta/Magister/Materias/1er Semestre/Astroinformatica I/Practice_1$ python3 task2.py 1
Your Star is a Brown Dwarf
jdg@jdgpc:~/Documents/Tareas_ID/Astrofagasta/Magister/Materias/1er Semestre/Astroinformatica I/Practice_1$ python3 task2.py 30000
The Spectral Type of your Star is: M
jdg@jdgpc:~/Documents/Tareas_ID/Astrofagasta/Magister/Materias/1er Semestre/Astroinformatica I/Practice_1$ python3 task2.py 10000
The Spectral Type of your Star is: A
jdg@jdgpc:~/Documents/Tareas_ID/Astrofagasta/Magister/Materias/1er Semestre/Astroinformatica I/Practice_1$ python3 task2.py 50000
The Spectral Type of your Star is: O
jdg@jdgpc:~/Documents/Tareas_ID/Astrofagasta/Magister/Materias/1er Semestre/Astroinformatica I/Practice_1$ python3 task2.py 1000000
Your star may be a Wolf-Rayet Star (TOO HOT!)

```

Figure 4. Output of the program for different temperature inputs.

III. TASK 3

The final task from this practice focused on writing a Shell Script capable of calculating the Julian Date for a given date on the Gregorian Calendar (the commonly used one). To achieve this, the following equation was given:

$$Julian = \frac{36525 \cdot year}{100} + \frac{306001 \cdot (month + 1)}{10000} + day + 1720981 \quad (1)$$

Also, the following considerations were given in order to use this equation correctly:

1. The month is 13 for January.
2. The month is 14 for February.
3. The month is 3 for March, and so on.
4. For January and February the year is reduced by 1.

The following Shell Script was used to achieve this task:

```

echo -n "Enter_day:_ "
read day
echo -n "Enter_month_(1-12):_"
read month
echo -n "Enter_year:_ "
read year

# Adjust January and February weird months
if (( month == 1 )); then
    month=13
    ((year--)) #Reduction of year by 1

```

```

elif (( month == 2 )); then
    month=14
    ((year--)) #Same Here
fi

# Julian day formula
julian=$(( (36525 * year) / 100 \
    + (306001 * (month + 1)) / 10000
    + day + 1720981 ))

echo "Julian_day=$_julian"

```

As we can see, the shell script asks for the Gregorian

date as an input (a set of 3 integer variables), performs the necessary modifications to the values if the month given was January or February, and finally it uses the formula to calculate the Julian Day. We can see the code in action in the following Screen-shot:



```

root@jaguar:~/Documents/Tareas_3D/Astrofagasta/Register/Materias/1er Semestre/AstroInformatica I/Practice_2$ ./task3.sh
Enter day: 7
Enter month (1-12): 6
Enter year: 2008
Julian day = 2454624

```

Figure 5. Calculation of the Julian Day for the Gregorian date: 7 Jun 2008.